

次に、市場データの「どの価格を使うか」を決めます。配当、株式分割、為替の影響を混ぜると、同じ予想でも結果が変わるためです。

予想精度と投資成績で指標を分ける

予想精度

アナリストが予想した対象そのものの値動きを評価します。

- 株価予想：現地通貨建ての株価
- 日経平均予想：日経平均株価
- ドル円予想：USD/JPY
- 原油価格予想：発言で指定された原油価格または確定済み代理指標

投資利用成績

実際に投資した場合の結果を評価します。

- 配当を含むリターン
- 日本円換算後のリターン
- ベンチマークと比較した超過リターン

例えば米国株がドル建てで+10%でも、同期間に円高が進めば、日本円での投資成績はマイナスになる可能性があります。

株式について保存する結果

指標	用途
現地通貨建て株価リターン	アナリストの方向予想評価
配当込み現地通貨リターン	現地投資家としての投資成績
日本円換算後の配当込みリターン	ユーザーにとっての投資利用成績
ベンチマーク超過リターン	銘柄選択能力の評価

基準通貨は初期値を日本円とし、将来はCLI入力に変更できるようにします。

株式分割の扱い

例えば、予想時点で株価10,000円、目標価格12,000円だった銘柄が、その後1対2の株式分割を行った場合、分割後の目標相当価格は6,000円です。

したがって、

- 方向性評価：株式分割調整後価格を使用
- 目標価格評価：予想時点の目標価格を分割比率に合わせて調整
- 原文の「12,000円」はそのまま保存
- 評価用目標価格として「分割後換算6,000円」を別に保存

とします。

配当の扱い

「株価が上昇する」という予想は、原則として配当を含めず株価だけで評価します。

「買い」「投資すべき」という推奨は、

- 株価リターン
- 配当込み総合リターン

の両方を評価します。

データ取得元に配当情報がない場合は、

```
total_return_status: unavailable
```



とし、株価リターンを配当込み成績であるかのようには扱いません。

指数の扱い

「日経平均が上昇する」のような予想は、発言で指定された価格指数を主評価にします。

投資利用成績については、利用可能であれば、

- 価格指数
- 配当込み指数
- 対応するETFの投資成績

を分けて表示します。

指数とETFでは信託報酬、配当、連動誤差などがあるため、同一の数値として扱いません。

ベンチマーク比較

ベンチマークと対象は、同じ条件にそろえます。

- 同じ評価期間
- 同じ基準通貨
- 両方とも株価リターン、または両方とも総合リターン
- 同じ開始価格ルール
- 同じ営業日調整

例えば日本円換算した米国株と比較する場合、ベンチマークも日本円換算します。

MVPで保存する3種類の結果

1. 予想対象自体の値動き
2. 実際に投資した場合の総合リターン
3. ベンチマークに対する超過リターン



これらはすべてPythonで計算し、データが不足する項目だけ「評価不能」とします。

この方針であれば、

アナリストの株価方向予想は当たったが、日本の投資家にとって有利な投資ではなかった

という違いも判定できます。初期の基準通貨を日本円とする方針を含め、この方法を提案します。

OK

次に、「時期」と「程度」の評価方法を定めます。

時期評価

時期については、発言者自身がどこまで明言したかを重視します。

時期の由来	実績計算	アナリストの時期精度
発言者が明示	実施する	評価対象
文脈からAIが推定	実施する	原則として参考値
システム既定の1・3・6・12か月	実施する	時期精度には含めない
時期情報なし	実施する	評価対象外

AIやシステムが設定した期限を外したという理由で、アナリストの時期精度を下げない設計にします。

時期の評価結果

時期を明示した予想には、次を保存します。

- 予想開始日
- 予想終了日
- 許容期間
- 実際の達成日
- 予想期間内に達成したか
- 予想期間から何日ずれたか
- 時期の二値判定
- 時期の連続スコア

例えば、

予想：9月中に45,000円へ到達
実績：10月8日に到達

期間内判定：不的中
予想期間からのずれ：8日
時期スコア：比較的高い

とします。

つまり、8日外れた予想と半年外れた予想を、同じ0点にはしません。具体的な連続スコアの減点

式は、テストデータを確認してから固定します。

数値で示された程度

数値予想は、発言表現によって評価方法を分けます。

発言	数値の意味	評価
20%上昇する	おおむね 20%	実際の変化率との 誤差
少なくとも20% 上昇する	下限	20%以上なら達成
最大20%程度上 昇する	上限	上限との関係の評価
10～20%上昇 する	範囲	範囲内なら達成
45,000円になる	目標値	実際の価格との誤差
45,000円を超える	閾値	超えたかを評価
45,000円に一時 到達する	到達値	期間中高値で評価

そのため、構造化データには数値だけでなく、次を保存します。

```
magnitude_operator:
  approximate
  minimum
  maximum
  range
  exact_target
  threshold
```



数値予想の出力

例えば「3か月後に20%上昇する」に対して実績が+12%なら、

方向：的中
予想変化率：+20%
実際の変化率：+12%
絶対誤差：8ポイント
予想値に対する誤差率：40%
程度判定：部分的中



とします。

方向が当たっていても、程度まで完全に当たったことにはしません。

定性的な程度

「大幅に上昇する」「少し下落する」などは、AIが勝手に具体的なパーセントへ変換しません。

保存する項目は次のとおりです。

原文表現：大幅に上昇する
定性的程度：large
数値への変換：なし
実際の変化率：+4.2%
ボラティリティ調整後変化：+0.3



MVPでは、

- 方向の的中・不的中
- 実際に何%動いたか
- 通常の値動きと比べてどの程度だったか

を表示します。

「大幅」という表現が本当に適切だったかの正式な程度スコアは、資産種別や予想期間別の基準を検証してから導入します。それまでは補助判定にとどめます。

MVPの採点方針

必ず採点する

- 明示された方向
- 明示された時期
- 明示された数値・範囲・閾値

実績は計算するが、正式採点しない

- AIが推定した時期
- システム既定期間
- 「大幅」「少し」などの定性的程度

これにより、発言者が実際に言っていない数値や期限で不当に評価することを防ぎます。この方針を提案します。

OK

次に、条件付き予想と確率付きシナリオの扱いを定めます。

条件付き予想

例えば、

日銀が追加利上げを行えば、円高になる

という発言は、次の2段階に分けます。

1. 条件が成立したか
2. 条件成立後に予想結果が発生したか

評価状態

```
condition_not_met
condition_met
condition_partially_met
condition_unclear
outcome_pending
outcome_evaluated
```



条件が成立しなかった場合は不的中ではなく、評価対象外とします。

```
条件：日銀が追加利上げする
実績：追加利上げなし
判定：条件不成立
的中率への算入：なし
```



条件成立後の評価開始

条件成立後に結果が起きるという予想なら、条件が成立した時点から評価します。

発言日：7月1日
条件成立：9月1日
予想結果：条件成立後3か月以内に円高
評価期間：9月1日から12月1日

📄

ただし、「年末までに利上げされれば円高になる」のように全体の期限がある場合は、その期限も維持します。

条件自体も予想している場合

次の発言は二つの予想を含みます。

日銀は9月に利上げし、その結果として円高になる。

予想1：9月に日銀が利上げする
予想2：利上げ後に円高になる

📄

両方を親子関係付きで評価します。

一方、

もし日銀が利上げすれば円高になる

だけなら、日銀が利上げすると予想したことにはしません。

条件成立の判定

条件によって担当を分けます。

条件	判定方法
株価が一定価格を超える	Python
指数が一定率下落する	Python
政策金利が変更される	公的データ+Python
選挙結果、政策決定	AIによる資料確認

条件	判定方法
景気が悪化する	定義が曖昧なためAI判定・ダブルチェック
地政学リスクが高まる	原則として曖昧・評価困難

曖昧な条件をAIが無理に成立・不成立へ決めず、`condition_unclear`を許容します。

複数条件

次のような論理関係も保存します。

```
condition_logic:
  AND
  OR
  NOT
```



例えば、

利下げが行われ、かつ景気後退が回避されれば株価は上昇する

では、両方が成立した場合だけ結果を評価します。

確率付きシナリオ

例えば、

60%の確率で上昇、30%で横ばい、10%で下落

の場合は、全シナリオと確率を保存します。

- 最も確率が高いシナリオを主要予想として方向的中率へ算入
- 他のシナリオも削除しない
- 確率の合計を検査する
- 長期的には予想確率と実際の発生率を比較する

MVPでは、主要シナリオの方向評価を行います。確率予想の精度評価やキャリブレーションは、十分な件数が蓄積してから実装しますが、必要な確率データは最初から保存します。

本命を数値で示さない場合

上昇がメインシナリオだが、下落の可能性もある

の場合は、

- 上昇：主要予想
- 下落：代替シナリオ
- 確率：不明

として、上昇を方向的中率へ算入します。

上昇も下落もあり得る

だけで本命を示さない場合は、これまでの決定どおりの中率から除外します。

この方針により、条件が起きなかった予想を外れとせず、複数シナリオを述べただけで「どちらに動いても的中」となることも防げます。

OK

次に、過去成績を現在の予想へどう反映し、「上昇候補」を抽出するかを定めます。

分野・期間が近い過去成績を優先する

現在の予想を評価する際、アナリストの総合的中率をそのまま使用しません。

例えば現在の予想が、

ドル円は今後3か月で円安になる

であれば、過去成績を次の順で参照します。

1. ドル円・3か月程度の過去予想
2. 為替・3か月程度の過去予想
3. 為替全般の過去予想
4. 3か月程度の方向予想全般
5. アナリスト全体の方向予想
6. 十分なデータがなければ「実績不足」

完全一致する分類の件数が少ない場合は、順次広い分類を補助的に使用します。

少数実績の過大評価を防ぐ

2件中2件の中したアナリストを、そのまま的中率100%として高く評価しません。

過去件数が少ない場合は50%付近へ補正し、件数が増えるほど実際の的中率を強く反映させます。

アナリストA：2件中2件の中

単純的中率：100%

補正後信頼度：限定的



アナリストB：80件中56件の中

単純的中率：70%

補正後信頼度：比較的高い

具体的にはベイズ補正などをPythonで行います。使用する計算式と初期値は、実データで検証してから固定します。

繰り返し予想の影響

合意したとおり、同じ予想を後日再表明した場合は、それぞれ独立した予想としての的中率へ算入します。

ただし、同じ見通しを10回繰り返して10回の中した場合、それらは同じ市場変動によって同時に当たった可能性があります。

そのため、現在予想の信頼度計算では、

- 予想表明ベースの成績
- 予想グループ調整後の成績
- 固有の予想グループ数

を併用します。

個別の予想はすべて数えますが、同一グループの繰り返しによってサンプル数が10件分増えたとはみなしません。

現在予想の集約結果

予測対象ごとに、次を出力します。

予測対象：ドル円
評価期間：今後3か月



上昇・円安予想：4人
下落・円高予想：2人
中立・不明：1人

過去成績で重み付けした方向：円安寄り
定量予想の加重平均：1ドル＝○○円
現在有効な予想数：6件
実績十分なアナリスト：3人
実績不足のアナリスト：3人
見解の一致度：中

MVPで現在予想へ反映する要素

処理を複雑にしすぎないため、初期版の重み付けは次に限定します。

- 該当分野での過去方向成績
- 該当予想期間での過去方向成績
- サンプル数の補正
- 予想の具体性
- 現在も有効な予想か
- 発言からの経過時間

分析品質や修正能力は表示しますが、MVPでは現在予想の重み付けへ直接使用しません。主観的なAI評価が、肝心の過去の中実績を過度に上書きすることを防ぐためです。

上昇候補の表示

候補は単一順位だけでなく、次の情報とともに表示します。

項目	内容
通常の評価	上昇・下落を予想した人数
重み付き方向	過去成績を考慮した方向
予想変化量	数値予想がある場合の集計
予想期限	いつまでの予想か

項目	内容
一致度	アナリスト間の見解の一致
実績信頼度	過去サンプル数が十分か
代理指標依存度	テーマ等の評価対象が安定しているか
根拠	各アナリストの発言と原文リンク

同じアナリストが現在の同一見解を5回繰り返しても、賛成人数は1人です。最新の有効な見解を代表として使用し、再表明回数と継続期間は補助情報として表示します。

初期段階では行わないこと

- ・ 的中率50%未満のアナリストの予想を自動的に逆張り利用する
- ・ 分析品質スコアで大きく重みを変える
- ・ 市場環境別の成績を重みに加える
- ・ 他のアナリストの発言をコピーした可能性を補正する
- ・ 上位候補を自動的な売買推奨とみなす

これらは詳細説明付きで [FUTURE_ROADMAP.md](#) へ記録します。

この「近い分野・期間の過去成績を優先し、少数データと繰り返し予想を補正したうえで、現在予想を集約する」方針を提案します。

OK

次に、調査対象期間を一つだけ指定する現在案を見直す必要があります。

「直近6か月の発言を調べる」だけでは、1・3・6・12か月後に評価する予想の多くがまだ評価期

間中です。そのため、次の2種類の期間を分けることを提案します。

1. 過去実績を作る期間

アナリストの得意分野や的中率を計算するために、過去の発言を収集する期間です。

目的：過去の予想能力を評価する
対象：評価期限を迎えた過去予想
例：過去3年間



期間指定のない予想は12か月後まで観測するため、直近1年以内の予想には未評価項目が多くなります。それ自体は問題なく、評価完了・評価中を分けます。

2. 現在予想を収集する期間

今後上昇しそうな対象を探すため、現在も有効な予想を収集する期間です。

目的：現在有効な見解を抽出する
対象：直近の発言
初期値：過去6か月



当初想定されていた「今日から過去6か月」は、こちらの初期値として維持します。

実行モード

CLI起動時に、次の3モードから選べるようにします。

過去実績作成

過去の予想を収集・評価し、アナリスト能力プロファイルを作ります。

モード：過去実績作成
分析対象者：○○
発言収集期間：2023-01-01～2026-07-20
評価基準日：2026-07-20



現在予想収集

現在有効な予想を収集し、既存の能力プロファイルを使って上昇候補を抽出します。

モード：現在予想収集



分析対象者：○○

現在予想の探索期間：過去6か月

基準日：2026-07-20

一括実行

過去実績と現在予想の両方を処理します。

保存する日付

各分析案件に、次を明確に保存します。

```
source_period_start
source_period_end
current_forecast_lookback
evaluation_as_of
market_data_cutoff
run_executed_at
```



これにより、「どの時点の情報で、どこまで結果が判明していたか」を再現できます。

現在予想へ使う過去成績

2026年7月20日時点で現在予想を評価する場合は、同日までに評価結果が確定している過去予想だけを使用します。

評価完了済み：能力計算に使用



評価期間中：件数は表示するが能力計算には未使用

条件未成立：的中率から除外

評価不能：理由を表示して除外

過去時点の再現

将来的に、

2025年1月1日時点でこの仕組みを使っていたら、どの銘柄が上位になっていたか

を検証する場合は、2025年1月1日までに判明していた情報だけを使用します。

これは将来のバックテスト機能として、詳細を `FUTURE_ROADMAP.md` へ残します。ただし、必要な日時情報はMVPから保存します。

推奨する初期値

- 現在予想の探索期間：過去6か月
- 過去実績の収集期間：過去3年
- 評価基準日：実行日
- 特定の銘柄・テーマ等：任意入力
- 期間はすべてCLIで変更可能

過去3年は暫定値です。十分な予想件数が得られない場合は5年などに広げられる設計にします。

このように「過去実績作成」と「現在予想収集」を分けることで、直近予想ばかりでの的中率を計算できない問題を避けられます。

過去実績を作る期間と現在予想を収集する期間を分離すると一覧性が損なわれてしまうので、それはやめて欲しい。加えて、評価機関が終了していない予想であっても、既にその予想通りの結果が発生している可能性もあり、評価を行う前にこの2つを分離することはできない。したがって予想情報は見境なく修正し、まだ過去実績を評価する。まだ評価期間が満了していない情報を用いて現在予想とするというアプローチはどうでしょうか。

そのアプローチの方が自然です。前回提案した「過去実績用」と「現在予想用」に収集段階から分離する方式は撤回します。

予想は指定期間から一括収集し、評価基準日時点の状態によって、過去実績・現在予想・評価中などへ自動分類する設計に変更します。

統合した処理方法



収集時点では用途を分けません。同じ予想データから、必要に応じて「過去実績一覧」と「現在予想一覧」を生成します。

CLIの期間入力

入力する期間は一つに戻します。

分析対象者：
調査対象期間：
 開始日：
 終了日：実行日
評価基準日：実行日
特に抽出したい予測対象：
調査媒体：



調査対象期間の初期値は、当初案どおり過去6か月とします。

ただし、過去成績の件数が少ない場合は、

評価完了済み予想が少ないため、信頼できる能力評価には調査期間の拡大を推奨

と表示します。期間を自動的に変更することはありません。

評価期間中でも途中経過を評価する

ご指摘どおり、評価期間満了前でも予想がすでに実現している場合があります。

ただし、「早期に的中が確定する予想」と「終了日まで結果が確定しない予想」を区別します。

早期確定できる予想

年末までに日経平均が45,000円へ一度到達する

10月に45,000円へ到達すれば、その時点で `fulfilled_early` としての的中確定です。その後下落しても、「一度到達する」という予想結果は変わりません。

早期確定できない予想

年末時点で日経平均が45,000円になる

10月に45,000円へ到達しても、年末価格はまだ分からないため、評価中です。

現在価格：45,000円
途中経過：予想値に到達
最終判定：評価期間中



方向予想

今後6か月で上昇する

3か月時点で上昇していても、その後下落する可能性があるため、最終的中とはしません。

状態：`active_on_track`
途中リターン：+12%
最終判定：未確定



一方、現在下落していても、期限までに上昇へ転じる可能性があるため、直ちに外れとはしません。

状態：`active_off_track`
途中リターン：-8%



予想状態

評価基準日時点で、各予想表明に次の状態を付けます。

状態	意味
<code>not_started</code>	予想の評価開始時期前
<code>active_on_track</code>	評価期間中で、現在は予想方向に進んでいる
<code>active_off_track</code>	評価期間中だが、現在は予想と反対方向
<code>active_indeterminate</code>	評価期間中だが、途中判定できない
<code>fulfilled_early</code>	期限前に的中が確定した
<code>failed_early</code>	期限前に不的中が確定した
<code>expired_hit</code>	評価期間終了後に的中確定
<code>expired_miss</code>	評価期間終了後に不的中確定
<code>condition_pending</code>	条件付き予想の条件待ち
<code>withdrawn</code>	本人が撤回した
<code>superseded</code>	後の予想に置き換えられた
<code>unevaluable</code>	データ不足・曖昧さ等で評価不能

現在予想として使う範囲

原則として次を現在予想一覧へ表示します。

- `not_started`
- `active_on_track`
- `active_off_track`
- `active_indeterminate`

- `condition_pending`

ただし、`not_started` と `condition_pending` は、通常の現在シグナルとは区分します。

`fulfilled_early` はすでに予想が実現しているため、現在の上昇予想へ自動的に加えません。「最近的中した予想」として別表示します。

`withdrawn` と `superseded` も現在予想から除外します。

一覧性の維持

データベースと一覧表は一つに統合します。

全予想一覧



- └ 状態で絞り込み：評価完了
- └ 状態で絞り込み：現在有効
- └ 状態で絞り込み：途中経過
- └ 状態で絞り込み：条件待ち
- └ 状態で絞り込み：評価不能

予想情報を別のフォルダや別のテーブルへ移動するのではなく、評価基準日時点の状態を付けて表示を切り替えます。

再評価

同じ予想について、アプリを後日再実行すると市場データだけを追加取得し、状態を更新します。

8月時点：`active_on_track`



10月時点：`fulfilled_early`

過去の状態も履歴として残すため、「その時点ではどう見えていたか」を後から確認できます。

したがって、正式な方針は次のとおりです。

指定期間の予想を用途の区別なく一括収集する。評価基準日時点で、評価完了・評価期間中・早期確定・条件待ち等へ動的に分類し、同一データから過去実績と現在予想の両方を生成する。

OK。ただし、想定よりも早期に予想が実現した場合、的中とは言えないかもしれないが、一定の評価は与えられるようにしたい。すでに反映済みですか？

部分的には反映済みですが、明確な「早期実現の部分評価」としては不十分でした。

これまでの仕様には、

- 時期スコアを0か100ではなく連続評価する
- 期間中の最高値・最低値を保存する
- 予想期間から何日ずれたかを記録する

という要素があります。これを「早期実現評価」として正式に定義します。

発言表現によって扱いを分ける

予想表現	早期実現の扱い
年末までに45,000円へ到達	早くても完全的中
年末時点で45,000円	早期到達は部分評価。最終判定は年末価格
12月頃に45,000円	早すぎる場合は時期減点付きの部分的中
12月から下落する	早期下落は方向評価を与えるが時期を減点
今後6か月で上昇する	途中上昇を記録するが、最終方向は6か月後に確定
12月中は高値圏を維持	早期到達だけでは的中にならず、維持状態を評価

「までに」は期限なので早期実現を減点しません。「時点で」「頃に」「から」は時期自体が予想内容なので、早すぎる場合は完全的中にはしません。

早期実現で保存する情報

- 予想された時期
- 最初に予想内容が実現した日
- 予想時期より何日早かったか
- 早期実現時の価格・変化率
- 実現状態が何日継続したか
- 予想時点で再び実現したか
- 予想期限時点の最終結果
- 方向スコア
- 程度スコア
- 時期スコア
- 早期実現フラグ

状態として以下を追加します。

```
pre_window_realization
early_temporary_realization
early_sustained_realization
```



評価例

予想：

日経平均は12月末に45,000円になる

実績：

- 10月1日に45,000円へ到達
- 10月中旬に下落
- 12月末は42,000円

評価は次のようになります。

```
正式な時点予想：不的中
早期実現：あり
最初の到達日：10月1日
予想時期との差：91日早い
程度の部分評価：あり
時期評価：減点
12月末の価格誤差：-3,000円
```



単純に「完全な外れ」とはせず、

予想した価格水準には到達したが、時期が大幅に早く、その水準を予想時点まで維持できなかった

と評価できます。

上昇後に下落した場合

予想：

今後6か月で20%上昇する

実績：

- 3か月後に+25%
- 6か月後に−5%

「6か月後に20%上昇」という時点予想なら、正式結果は不的中です。ただし、

期間中最大上昇：+25%
目標水準への早期到達：あり
目標到達まで：3か月
終了時点：−5%
早期実現評価：あり
最終評価：不的中



とします。

一方、「6か月以内に20%上昇する」なら、3か月後の+25%で完全的中です。

スコアの考え方

一つの的中・不的中だけで処理せず、次を分離します。

正式判定 ： 的中／不的中
方向スコア ： 0～100
程度スコア ： 0～100
時期スコア ： 0～100
早期実現評価 ： 0～100



ただし、「早期実現評価」を総合点へ何%加えるかは、実際のデータを確認してから決めます。

MVPでは各項目を個別表示し、恣意的な重み付けを避けます。

また、早期に一度実現した後、予想期間内にも再び実現した場合は正式な中として扱います。早期実現と期間内実現を2件の中として重複計上はしません。

したがって、正式方針は次のようになります。

予想時期より早く予想内容が実現した場合、発言表現上「期限内」を意味する予想なら完全の中とする。特定時点・特定期間を意味する予想なら、正式判定とは別に、方向・程度・時期・継続性に基づく早期実現評価を与える。

OK

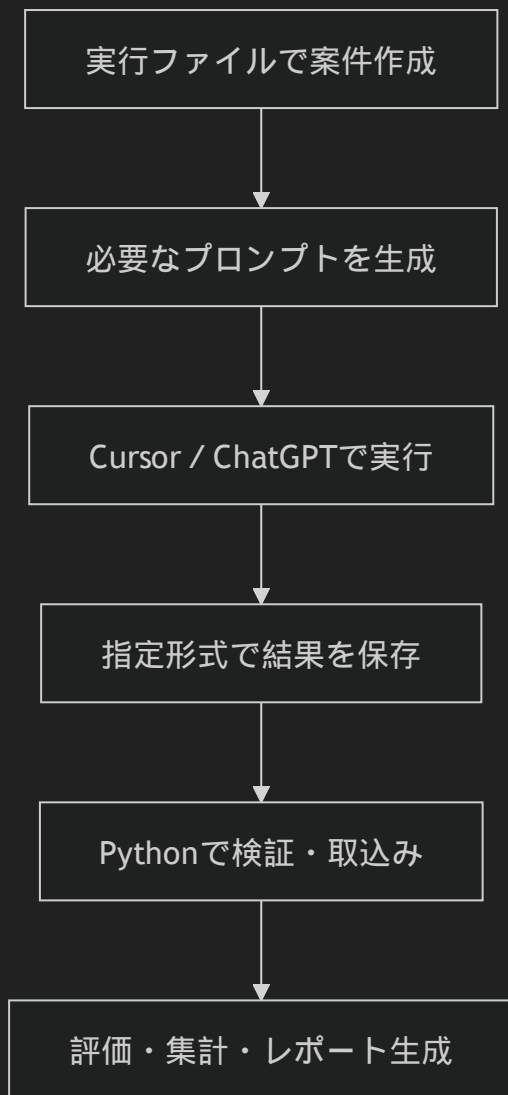
これまでで評価方法の中核はかなり固まりました。次は、ターミナル・Cursor・ChatGPTをどう連携させるかを定めます。

基本方式

初期版は、Python製の実行ファイルが全体を管理し、AIを使う工程だけプロンプトファイルを生成する半自動方式にします。

Mermaid

</> ▶ ↺



実行ファイルの初期画面

1. 新しい分析案件を作成
2. 既存案件の続きを処理
3. AI出力を検証・取り込む
4. 市場実績を更新する
5. レポートを再生成する
6. 案件の進捗を確認する
7. 終了



新しい分析案件

以下を入力します。

- 分析対象者
- 調査対象期間
- 評価基準日
- YouTubeを調査するか
- ブログを調査するか

- Xを調査するか
- その他Web記事を調査するか
- 特に抽出してほしい銘柄・テーマ等
- 使用するAI環境：Cursor／ChatGPT／両方

案件内の進捗管理

案件ごとに、各処理の状態を保存します。

未開始
プロンプト生成済み
AI実行待ち
AI出力あり
形式エラー
ダブルチェック待ち
検証完了
Python評価待ち
評価中
完了



ターミナルでは次のように表示します。

情報源探索／YouTube	完了
YouTube原文整理	3件中2件完了
話者推定レビュー	1件待ち
ブログ情報源探索	AI実行待ち
予想抽出	12件中12件完了
予測対象解決	4件待ち
市場実績評価	8件完了、4件評価期間中
レポート生成	未実行



プロンプトファイルの構成

各プロンプトは、当初のご希望どおり別々のMarkdownファイルにします。

</> Markdown



プロンプト名

目的

この処理で何を行うか。

使用するタイミング

どの処理が完了した後に実行するか。

推奨AI環境

CursorまたはChatGPT。

推奨モデル
高性能な推論モデル。

入力ファイル
読み込ませるファイルとフォルダ。

出力先
保存するファイル名と形式。

注意事項
原文変更禁止、日付制限、後知恵禁止など。

依頼文
ここから下をそのままコピーして使用する。

モデル名は設定ファイルから差し込みます。モデルが将来変更された場合でも、プロンプト本文を全面修正せずに済むようにします。

Cursorで実行する場合

Cursor用プロンプトには、読み込むフォルダと出力先を具体的に記載します。

入力：
analysts/A0001/RUN-0001/sources/youtube/ra

出力：
analysts/A0001/RUN-0001/sources/youtube/pr

Cursorには、指定先へJSONまたはMarkdownを直接保存させます。

ChatGPTで実行する場合

ChatGPTはObsidianフォルダへ直接保存できるとは限らないため、次のいずれかにします。

1. JSONファイルとして生成し、指定フォルダへ保存する
2. コードブロックでJSONを出力し、所定ファイルへ貼り付ける

ChatGPT用プロンプトにも、出力ファイル名を指定します。

出力ファイル名：
forecast_extraction_BATCH-0001.json

その後、実行ファイルが形式を検証して取り込みます。

プロンプトの正本と案件用コピー

prompts/

プロンプトの最新版



各分析案件/prompts_snapshot/

その案件で実際に使用したプロンプト

案件作成時には、中央の最新版から必要なプロンプトだけを生成します。各案件には、分析対象者名、期間、対象媒体、入出力先などを埋め込んだコピーを保存します。

これにより、

- プロンプトの改善を中央で管理できる
- 過去案件で実際に使った内容を再現できる
- Cursor用とChatGPT用を選べる
- 同じ案件の続きを後日実行できる

ようになります。

Pythonでの自動検証

AI出力を受け取った後、Pythonが次を検査します。

- JSON形式が正しいか
- 必須項目があるか
- IDが重複していないか
- 原文引用が実際の原文中に存在するか
- 日付形式が正しいか
- 発言日時より後の資料が混入していないか
- 参照ファイルが存在するか
- 同じ原文を再処理していないか

形式エラーの場合は、修正用プロンプトを自動生成します。

自動監視は後回し

MVPではフォルダを常時監視せず、ユーザーが実行ファイルから「AI出力を検証・取り込む」を選

択する方式にします。

将来的には、

- ファイル保存を検知して自動取込み
- Cursorのヘッドレス実行
- APIによるAI処理
- 全工程の一括実行
- 定期的な新規発言監視

へ拡張できるよう、`FUTURE_ROADMAP.md` に詳細を残します。

この方式なら、CursorとChatGPTを併用しながら、どこまで処理が終わっているかをターミナルで一元管理できます。

OK

次に、Obsidian内の保存構造を定めます。過去実績と現在予想は分離せず、一つの予想データから表示を切り替える方針を反映します。

推奨フォルダ構成

```
30_Permanent/
└─ ★アナリスト調査/
    │
    │ └─ _system/
    │   │ └─ config.yaml
    │   │ └─ database.sqlite
    │   │ └─ analysts.yaml
    │   │ └─ forecast_targets.yaml
    │   │ └─ target_mappings.yaml
    │   │ └─ prompt_versions.yaml
    │   │
    │   └─ docs/
    │     │ └─ PRODUCT_VISION.md
    │     │ └─ METHODOLOGY.md
    │     │ └─ DATA_MODEL.md
    │     │ └─ MVP_PLAN.md
    │     │ └─ FUTURE_ROADMAP.md
    │     │ └─ DECISION_LOG.md
    │     │ └─ OPEN_QUESTIONS.md
    │     │
    │     └─
```

```
└─ prompts/
|   └─ source_discovery/
|   └─ youtube_processing/
|   └─ forecast_extraction/
|   └─ target_resolution/
|   └─ validation/
|   └─ analysis/
|   └─ reporting/
|
└─ analysts/
    └─ analyst-name__A0001/
        └─ analyst_profile.md
        └─ RUN-20260720-001__20260120_202
            └─ request.yaml
            └─ status.yaml
            └─ README.md
            |
            └─ 01_prompts/
                └─ prompts_snapshot/
                    |
                    |
                    └─ 02_sources/
                        └─ youtube/
                            └─ raw/
                            └─ processed/
                                └─ metadata/
                        └─ blog/
                            └─ raw/
                            └─ processed/
                                └─ metadata/
                        └─ x/
                            └─ raw/
                            └─ processed/
                                └─ metadata/
                        └─ web/
                            └─ raw/
                            └─ processed/
                                └─ metadata/
                    |
                    └─ 03_ai_outputs/
                        └─ inbox/
                        └─ accepted/
                        └─ needs_review/
                        └─ rejected/
                    |
                    └─ 04_results/
                        └─ forecasts/
                        └─ evaluations/
                        └─ target_mappings/
                        └─ tables/
                        └─ charts/
                        └─ reports/
```

```
└─ 05_audit/
   └─ search_logs/
   └─ processing_logs/
   └─ evaluation_snapshots/
   └─ errors/
```

`raw` と `processed`

`raw`

取得した状態の原文を保存し、変更しません。

- YouTube文字起こし
- ブログ本文
- X投稿
- 記事本文
- 取得時のHTMLやテキスト
- ユーザーが手動で用意した文字起こし

`processed`

AIによる整理結果を保存します。

- 句読点の補完
- 段落分割
- YouTubeの話者推定
- 明らかな文字起こし誤りの候補
- ノイズ部分の識別
- 原文位置との対応情報

`processed` の内容から、必ず `raw` の該当箇所へ戻れるようにします。

SQLiteとMarkdownの役割

SQLite

機械処理上の正本です。

- 情報源
- 予想表明
- 予想グループ
- 予測対象
- 評価結果
- 評価履歴
- 現在状態
- アナリスト成績
- 検索ログ

- プロンプトバージョン

Markdown

人間がObsidianで読むための出力です。

- アナリスト概要
- 全予想一覧
- 現在有効な予想
- 評価完了予想
- 得意・不得意分野
- 分析品質
- 上昇候補
- 評価上の注意事項

Markdownを修正しても、原則としてSQLiteへ自動反映しません。データ修正は専用処理を通し、変更履歴を残します。

AI出力の取込み

ChatGPTやCursorの出力は、最初に

`03_ai_outputs/inbox/` へ入れます。

Pythonの検証後に、

- 正常： `accepted`
- AIによる再確認が必要： `needs_review`
- 使用不能： `rejected`

へ分類します。

正常データだけをSQLiteへ取り込みます。

結果フォルダを分離しすぎない

`04_results/forecasts/` には、過去予想と現在予想を分けず、統一された予想一覧を出力します。

各予想には現在状態を付けます。

```
active_on_track
active_off_track
fulfilled_early
expired_hit
expired_miss
```



次のファイルは、同じデータから生成する表示用レポートです。

```
all_forecasts.md
active_forecasts.md
completed_forecasts.md
forecast_evaluation_progress.md
```



したがって、現在予想と過去実績が別々の正本になることはありません。

IDとファイル名

分析対象者名だけを識別子にせず、固定IDを付けます。

```
analyst-name__A0001
```



改名や同姓同名があっても、同じ人物を追跡できます。

情報源にもIDを付けます。

```
SRC-000001__2026-07-01__youtube.txt
SRC-000002__2026-07-03__blog.md
```



予想表明はファイル名ではなく、SQLite内の固定IDで管理します。

```
FCI-000001
FCI-000002
```



Windowsの使用禁止文字やパス長問題を避けるため、ファイル名には短いIDと日付を使用し、長い動画タイトルや記事タイトルはメタデータ内へ保存します。

案件をまたぐ重複

同じ情報源が複数案件で発見された場合でも、原文ハッシュとURLを使って重複判定します。

- 原文データ自体は再処理しない
- どの案件で使用されたかは関連付けとして保存

- 案件ごとのプロンプトと評価基準日は維持
- 後から案件を統合しても元の実行履歴は削除しない

この構成なら、Obsidianでの閲覧性、Pythonによる集計、AI処理の再現性を両立できます。

OK

次に、生成するAIプロンプトの全体構成を整理します。処理を細かく分けすぎず、一つのプロンプトに複数の重い判断を詰め込みすぎない構成にします。

プロンプト一覧

情報源収集

ID	プロンプト	役割
P01	YouTube情報源探索	対象動画のURL・公開日時・出演情報を探索
P02	ブログ情報源探索	公式ブログ等の記事を探査
P03	X情報源探索	指定期間の本人投稿を探査
P04	その他Web記事探索	インタビュー・寄稿・引用記事等を探査

媒体選択時に、該当するプロンプトだけ生成します。

原文整理

ID	プロンプト	役割
P05	YouTube原文 整理・話者推 定	文字起こし分割、 整形、話者推定
P06	YouTube話者 推定レビュー	不確実な話者推定 の再検証
P07	テキスト原文 整理	ブログ・X・Web 記事の段落整理、 引用者識別

P06は、話者確信度が低い場合や、重要な予想が抽出されそうな場合にだけ実行します。

予想抽出

ID	プロンプト	役割
P08	予想抽出・構 造化	原文から予想候補 を抽出し、固定 JSON化
P09	予想抽出レビ ュー	原文にない補完、 否定表現、話者誤 認等を検証
P10	複合予想分解	上昇後下落などを 親予想・構成予想 へ分解

通常の見込みはP08だけで構造化します。P10は複数段階の見込みが含まれる場合だけ使用します。

予測対象解決

ID	プロンプト	役割
P11	予測対象解決	銘柄、指数、テー マ、業種等を正規 化
P12	予測対象解決 レビュー	後知恵や不適切な 代理指標を検証

ID	プロンプト	役割
P13	予測対象裁定	P11とP12の不一致を解決

P13は不一致案件だけに使用します。

重複・更新関係

ID	プロンプト	役割
P14	予想関係整理	再表明、転載、修正、反転、撤回等を分類
P15	分析バージョン整理	同じ根拠か、新しい分析が追加されたかを判定

P14で、独立した予想表明として数えるか、単なる再配布かを判断します。

詳細評価

ID	プロンプト	役割
P16	条件成立判定	Pythonで判定できない条件の成立を調査
P17	分析品質評価	根拠、リスク、反対要因等を予想グループ単位で評価
P18	評価例外レビュー	Pythonだけでは判定できない予想を確認

P16～P18は必要な予想だけに実行します。

統合・レポート

ID	プロンプト	役割
P19	アナリスト能力評価	計算済み成績から得意・不得意分野

ID	プロンプト	役割
		を整理
P20	現在予想統合 レビュー	現在有効な予想の 意味や対立関係を 整理
P21	最終レポート 作成	集計結果と根拠引 用からレポートを 作成

Pythonが担当するため、プロンプトを作らない処理

- フォルダ作成
- ID発行
- 原文ハッシュ計算
- JSON Schema検証
- 重複URL検出
- 市場価格取得
- リターン計算
- 方向・程度・時期の機械採点
- 評価状態の更新
- SQLiteへの保存
- 案件間のデータ統合
- 表・CSV・グラフ生成
- 進捗状況表示

案件間のマージは、PythonがIDとハッシュを使って行います。意味的な競合がある場合だけ、P14またはP18へ回します。

実行順序

P01～P04：選択媒体の情報源探索

↓

P05～P07：原文整理

↓

P08：予想抽出

↓

P09・P10：必要なものだけ検証・分解

↓

P11：予測対象解決

↓



P12・P13：検証と不一致裁定



P14・P15：再表明・修正・分析バージョン整理



Python：市場実績評価・集計



P16～P18：Pythonで処理できない例外



P19～P21：能力評価・現在予想・レポート

Cursor用とChatGPT用

中央では同じ論理プロンプトを管理し、案件作成時に使用環境に応じたコピーを生成します。

P08_forecast_extraction_cursor.md



P08_forecast_extraction_chatgpt.md

Cursor版はファイルパスと保存指示を含みます。

ChatGPT版は、

- 添付するファイル
- コピペする依頼文
- 出力ファイル名
- JSONだけを出力する指示

を含みます。

両方を選択した場合でも、同じ処理を無条件で二重実行せず、ダブルチェック対象など役割を分けます。

モデル指定

すべて高性能モデルを使用する方針を反映し、各プロンプトには次を記載します。

必要能力：



- 長文読解
- 高精度な意味判断
- 構造化出力
- 原文忠実性

使用モデル：

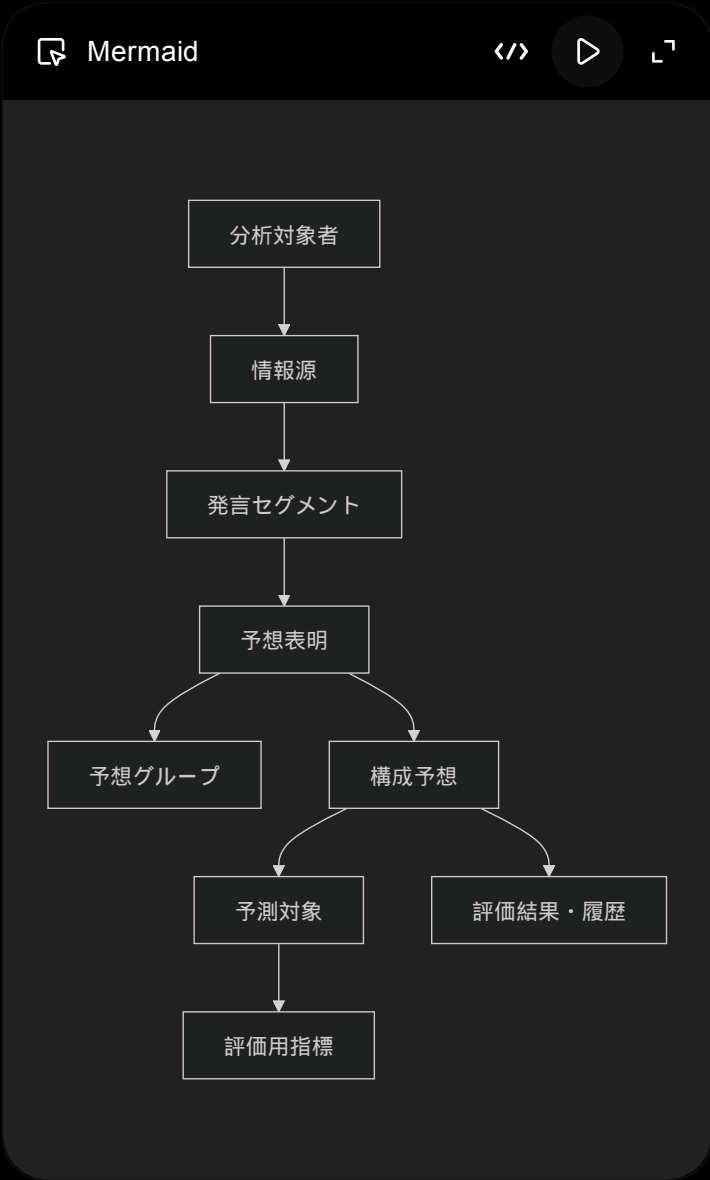
案件作成時の設定から挿入

モデル名だけでなく必要能力も残すため、将来モデルが入れ替わっても選び直せます。

この21種類は「毎回すべて実行するプロンプト数」ではありません。選択媒体や予想内容に応じて必要なものだけ生成・実行します。

OK

次に、データベースの基本構造を定めます。一つの巨大な表へ詰め込まず、「原文」「予想」「予測対象」「評価」を分離します。



1. 分析対象者

人物を固定IDで管理します。

analyst_id
氏名
別名・表記揺れ
所属
専門分野
公式YouTube
公式ブログ
公式x
プロフィール情報
登録日時



同姓同名や所属変更があっても、同一人物を維持します。

2. 情報源

動画・記事・投稿そのものを管理します。

source_id
analyst_id
媒体
URL
タイトル
チャンネル・サイト名
公開日時
収録日時
取得日時
証拠レベル
原文ファイル
原文ハッシュ
取得状態



一つのYouTube動画から複数の予想を抽出できます。

3. 発言セグメント

原文のどの部分を誰が発言したかを管理します。

segment_id
source_id
話者
話者推定確信度
タイムスタンプ
原文
整理済み文章
前後の文脈



原文内の位置

話者レビュー状態

予想表明から必ずこのセグメントへ戻れるようにします。

4. 予想表明

ある時点で改めて行われた一つの予想です。的中率の基本的なカウント単位になります。

forecast_issuance_id



analyst_id

source_id

segment_id

発言日時

公開日時

予想分類

方向

時期表現

数値表現

条件

確率

確信の強さ

証拠強度

抽出確信度

現在状態

同じ予想を後日改めて発言した場合は、別の

`forecast_issuance_id` を発行します。

5. 予想グループ

同じ見通しの継続・修正履歴をまとめます。

forecast_group_id



analyst_id

中心となる見通し

開始日

最新表明日

現在の見解

再表明回数

修正回数

撤回状態

各予想表明は個別評価しますが、繰り返しによる件数偏重の補正や、分析品質評価に使用します。

6. 構成予想

複合的な予想を分解した最小評価単位です。

例えば、

9月まで上昇し、その後年末まで下落する

を次に分けます。

構成予想1：現在から9月まで上昇

構成予想2：9月頃に天井

構成予想3：9月から年末まで下落



保存項目は次のとおりです。

forecast_component_id

forecast_issuance_id

親構成予想ID

順序

予想形式

方向

開始時期

終了時期

数値

数値演算子

条件ID



予想件数としては親の予想表明1件ですが、結果は各構成予想ごとに評価できます。

7. 予測対象

発言中の銘柄・テーマ・業種などを正規化します。

target_id

原文表現

正規名称

対象種別

銘柄コード

ティッカー

取引所

通貨

別名

有効期間



同じ対象について、

- トヨタ

- トヨタ自動車
- 7203
- Toyota Motor

などと同じ `target_id` へ関連付けます。

8. 評価用指標

テーマや業種を、Pythonで取得可能な指数・ETF・銘柄群へ対応付けます。

```
target_mapping_id
target_id
評価方法
使用ティッカー
構成銘柄
ウェイト
ベンチマーク
情報基準日
選定根拠
使用資料
AI提案結果
ダブルチェック結果
裁定結果
マッピングハッシュ
```



結果確認後にマッピングを書き換えません。変更
する場合は新しいバージョンを作ります。

9. 評価結果

各構成予想について、市場実績との比較結果を保存します。

```
evaluation_id
forecast_component_id
target_mapping_id
評価基準日
評価開始価格
現在価格
終了価格
期間中最高値
期間中最低値
実際の変化率
ベンチマーク変化率
超過リターン
方向判定
時期判定
程度判定
```



早期実現評価
評価状態
評価不能理由

10. 評価履歴

評価期間中の状態変化を残します。

```
evaluation_snapshot_id
evaluation_id
スナップショット日時
active_on_track
active_off_track
fulfilled_early
途中リターン
途中最高値
途中最低値
```



同じ予想が、

```
8月: active_off_track
10月: active_on_track
12月: expired_hit
```



と変化した履歴を確認できます。

派生データ

次は正本ではなく、SQLiteから再計算できる派生データとします。

- アナリスト別の中率
- 分野別成績
- 期間別成績
- 現在有効な予想一覧
- 上昇候補ランキング
- 予想グループ調整後成績
- グラフ
- Markdownレポート

計算方法を変更しても、原文や予想表明を再抽出せずに再生成できます。

具体例

情報源：
SRC-0001／7月1日のYouTube動画



発言セグメント：

SEG-0042／本人の発言とAI推定

予想表明：

FCI-0010／年末まで日経平均は上昇する

構成予想：

FCC-0012／7月から年末まで上昇

予測対象：

TGT-0003／日経平均株価

評価用指標：

MAP-0003／日経平均株価指数

評価結果：

EVAL-0012／年末時点の方向、変化率、途中経過

この構造なら、原文の証拠を維持しながら、同じ予想の再表明、複合予想、テーマ評価、途中経過、現在予想を一貫して処理できます。

OK、ただ、アプリケーションが非常に複雑なので、進捗状況から次に何をすればよいかがターミナルに表示されるようにしたい。加えて、Cursorに「次何すべき？」や「次の行為にどんな選択肢がある？」という質問をすれば、フォルダ内部を分析してつぎにやるべき行為を判断できるような状態にしたい。

また、何がAIプロンプトで何がpythonなのかが既に分からなくなりつつある。優しい説明書を準備するようにしてほしい。具体的には手順書兼取扱説明書、仕様書（フォルダ構造図、フローチャート等を含む、各機能の説明等仕様書によくある情報をを含む）、Readme（取扱説明書・仕様

書の簡易版)、スタートアップガイドをフォルダを作って入れて欲しい。

その要件は非常に重要です。ユーザーが21種類のプロンプトや処理順序を覚える設計にはしません。

ターミナルを見れば次に何をすべきかわかり、Cursorに質問しても同じ答えが得られる

ことを正式な機能要件に追加します。

ターミナルの「次にすること」機能

実行ファイルのメインメニューに、次を追加します。

1. 新しい分析案件を作成
2. 次にすることを表示
3. 推奨処理を続行
4. AI出力を取り込む
5. 市場実績を更新
6. 進捗一覧を表示
7. エラー・未解決事項を表示
8. レポートを開く
9. 終了



また、何らかの処理が完了するたびに、次の推奨行動を自動表示します。

表示イメージ

分析案件：RUN-20260720-001
分析対象者：○○
全体進捗：第4段階／全9段階



完了：

- ✓ YouTube原文取込み
- ✓ YouTube原文整理
- ✓ 話者推定
- ✓ 予想抽出

現在の状態：

抽出した予想12件のうち、3件に予測対象の曖昧さがあ

次にすること【推奨】：

[AI・Cursor]

予測対象解決プロンプトを実行してください。

プロンプト：

01_prompts/P11_target_resolution_cursor.mc

入力：

03_ai_outputs/accepted/forecast_extractor

出力先：

03_ai_outputs/inbox/target_resolution_BATC

この処理が必要な理由：

「AI関連株」など、Pythonが価格を取得する対象がま

完了後：

この実行ファイルで「AI出力を取り込む」を選択して、

その他の選択肢：

1. ChatGPTで同じ処理を行う
2. 先に別の媒体の原文を追加する
3. 対象解決を保留して、処理可能な9件だけ評価する
4. 詳細な進捗状況を表示する

誰が行う処理かを常に表示する

すべての処理に、次の分類を付けます。

表示	意味
[PYTHON・自動]	実行ファイルが自動処理
[AI・Cursor]	Cursorへプロンプトを入力
[AI・ChatGPT]	ChatGPTへプロンプトを入力
[AI・選択可能]	CursorまたはChatGPT
[ユーザー]	原文ファイルの準備など
[待機]	評価期間終了・条件成立等を待つ
[エラー対応]	修正または再実行が必要

プロンプト一覧を覚えなくても、ターミナルが使用するファイルを一つ指定します。

Python処理は可能な限り自動で進める

次のPython処理は、前提条件がそろった時点で連続実行できます。

- AI出力の形式検証
- SQLiteへの取込み
- 重複検出
- 状態更新
- 市場価格取得
- 計算可能な予想の評価
- 表とグラフの更新
- 次に必要なプロンプトの生成
- 進捗ファイルの更新

例えばAI出力を取り込んだ後、Pythonだけで進められる処理は自動実行し、次にAIまたはユーザーの操作が必要になった時点で停止します。

実行結果は必ず、

Pythonが自動実行した処理：5件
次にユーザーが行う処理：1件



と説明します。

次の行為を決める仕組み

Python側に処理順序を定義したタスクカタログを持たせます。

YAML



```
task_id: P11
name: 予測対象解決
executor: AI_CURSOR_OR_CHATGPT
prerequisites:
  - forecast_extraction_completed
inputs:
  - forecast_extraction_json
outputs:
  - target_resolution_json
next_tasks:
  - P12
alternative_tasks:
  - add_more_sources
  - defer_unresolved_forecasts
```


ターミナルは、タスクカタログと実際のファイル・データベース状態を比較して、次の行動を判断します。

AIに判断を丸投げせず、Pythonが出した進捗状態を正本にします。

Cursorへ「次何すべき？」と質問できる状態

各案件に、次のファイルを自動生成します。

```
README.md
status.yaml
WORKFLOW_STATE.json
NEXT_ACTIONS.md
OPEN_ISSUES.md
```



NEXT_ACTIONS.md

次の内容を常に最新状態で出力します。

- 現在の処理段階
- 完了した処理
- 次の推奨行動
- 推奨理由
- 使用するプロンプト
- 入力ファイル
- 出力先
- 代替手段
- ブロッカー
- 処理を保留した場合の影響

Cursorへの案内ファイル

プロジェクト直下に、Cursorが最初に読むための案内を置きます。

```
AI_WORK_GUIDE.md
```



このファイルには、Cursorへ次のように指示します。

ユーザーから「次に何をすべきか」「どのような選択肢と質問された場合は、最初に以下を確認してください。



1. 対象案件のstatus.yaml
2. WORKFLOW_STATE.json
3. NEXT_ACTIONS.md
4. OPEN_ISSUES.md
5. request.yaml

Pythonが生成した進捗状態を正本とし、存在しない処推測して完了扱いしないでください。

回答には、推奨行動、理由、実行者、使用プロンプト、入力、出力先、代替案を含めてください。

さらに、専用プロンプトを追加します。

P00_next_action_advisor_cursor.md



これをCursorへ貼り付けると、案件フォルダを確認して、

- 次に何をすべきか
- 今できる選択肢
- どれがおすすめか
- 各選択肢の影響
- エラーや不足ファイル

を回答します。

用意する説明書

プロジェクト直下とdocsフォルダに、次を作成します。

★アナリスト調査/



```
├─ README.md
└─ docs/
    ├─ 01_スタートアップガイド/
    │   └─ STARTUP_GUIDE.md
    ├─ 02_取扱説明書/
    │   └─ USER_MANUAL.md
    ├─ 03_仕様書/
    │   ├─ SYSTEM_SPECIFICATION.md
    │   ├─ WORKFLOW_DIAGRAMS.md
    │   ├─ DATA_MODEL.md
    │   └─ FOLDER_STRUCTURE.md
    └─ 04_参考資料/
        ├─ PROMPT_CATALOG.md
        ├─ PYTHON_TASK_CATALOG.md
        ├─ GLOSSARY.md
        └─ TROUBLESHOOTING.md
```

README

最も短い案内です。次だけ読めば起動できる内容にします。

- このアプリでできること
- 最初に開くファイル
- 起動方法
- 新規案件の作成方法
- 「次にすること」の確認方法
- 困ったときに見る場所
- 詳細説明書へのリンク

スタートアップガイド

初回利用者向けに、最初の1案件を作るところまで説明します。

- 必要なソフト
- インストール
- Obsidian Vaultの設定
- APIキー等の設定
- 実行ファイルの起動
- 分析対象者と期間の入力
- 最初のプロンプト実行
- AI出力の保存
- Pythonへの取込み
- 最初のレポート確認

画面例とターミナル表示例を含めます。

手順書兼取扱説明書

日常的な操作を、実際の作業順に説明します。

- 新しい分析案件
- 既存案件の続行
- YouTube文字起こしの準備
- Cursorプロンプトの使用
- ChatGPTプロンプトの使用
- AI出力の取込み
- エラー修正
- 市場実績の更新
- レポートの再生成
- 案件のマージ
- 未解決予想の確認

- バックアップと復旧

各手順に、実行者を明記します。

この作業を行うもの：AI・Cursor



この作業を行うもの：Python・自動

この作業を行うもの：ユーザー

仕様書

一般的なシステム仕様書に必要な内容を含めます。

- 製品目的
- 対象ユーザー
- 対象範囲・対象外
- 用語定義
- 利用シナリオ
- 機能一覧
- 機能要件
- 非機能要件
- システム構成図
- 全体フローチャート
- 状態遷移図
- フォルダ構造図
- データモデル
- SQLiteテーブル概要
- AIとPythonの役割分担
- プロンプト構成
- 評価方法
- エラー処理
- ログ・監査
- テスト方針
- 受入条件
- 制約・既知の限界
- 将来課題

図はMermaidで作成し、Obsidian上で確認できるようにします。

「AIかPythonか」早見表

README、取扱説明書、仕様書のすべてに、次のような早見表を掲載します。

作業	担当
フォルダ作成	Python
情報源探索	AI
YouTube話者推定	AI
予想抽出	AI
原文引用の存在確認	Python
意味の誤読確認	AI
予測対象候補の選定	AI
市場価格取得	Python
的中計算	Python
曖昧な条件の判定	AI
集計・グラフ	Python
能力評価の文章化	AI
次の行動の機械判定	Python
次の行動の分かりやすい説明	PythonまたはCursor

この説明書群と「次にすること」機能を、補助機能ではなくMVPの必須要件に追加します。

ok

ここまでで主要な仕様方針が固まりました。次は、複雑さを抑えて確実に完成させるため、開発を段階分けします。

推奨する開発順序

第0段階：技術検証

本格実装前に、失敗すると全体へ影響する部分だけ小さく検証します。

- YouTube文字起こしから話者をAI推定できるか
- 原文から予想を正確にJSON化できるか
- Pythonで日本株・海外株・指数・為替の価格を取得できるか
- テーマから評価用銘柄をAIで決定・再検証できるか
- CursorとChatGPTの出力を同じ形式で取り込めるか

各検証は、実際の原文数件で成功条件を確認します。

第1段階：最小構成の一連動作

分析対象者1人、手動で用意した原文5～10件程度で、最初から最後まで動かします。

- 案件作成
- フォルダ・プロンプト生成
- 原文取込み
- 予想抽出
- 予測対象解決
- 市場価格取得
- 方向・時期・程度評価
- 現在状態判定
- Markdownレポート
- 「次にすること」の表示

情報源探索の網羅性より、処理の正しさを優先します。

第2段階：媒体別の収集処理

- YouTube動画探索
- YouTube話者推定
- ブログ探索
- X探索
- その他Web記事探索
- 検索ログ
- 調査網羅度表示

- 差分取込み

第3段階：複雑な予想への対応

- テーマ・業種の代理指標
- 複数銘柄バスケット
- 条件付き予想
- 複合予想
- 再表明・修正・反転
- 早期実現
- 複数代理指標の感度分析

第4段階：複数アナリストの統合

- 分野・期間別能力プロファイル
- 少数サンプル補正
- 予想グループ補正
- 現在見解の統合
- 上昇候補一覧
- アナリスト横並び比較
- タイムライン・ヒートマップ

第5段階：品質向上

- 分析品質
- 修正能力
- 詳細なダブルチェック
- 評価式の検証
- エラー回復
- テスト拡充
- 操作説明書の改善

第5段階の機能も完成版では必須ですが、予想の抽出・市場評価が安定した後に詳細化します。

MVPの必須完成条件

最初のMVPは、次をすべて満たした時点で完成とします。

- 実行ファイルから案件を作成できる
- 指定したObsidianフォルダへ必要な構造が作られる
- 使用すべきプロンプトが案件別に生成される
- 原文を変更せず保存できる
- 原文引用付きで予想を抽出できる

- AI出力の形式をPythonで検証できる
- 既知の銘柄・指数をPythonで評価できる
- 評価中・的中・不的中・評価不能を区別できる
- 同じ予想の再表明を個別評価できる
- 全予想・現在予想・評価完了予想を同一データから表示できる
- ターミナルに次の推奨行動が表示される
- Cursorへ「次に何をすべきか」と質問できる
- README、スタートアップガイド、取扱説明書、仕様書がある
- 処理履歴と使用プロンプトを追跡できる
- 将来課題が十分な説明付きで記録される

最初の試験データ

最初から数百件を処理せず、次を含む少数の原文を選びます。

- 明確な上昇予想
- 数値・期限付き予想
- 期間不明の予想
- 条件付き予想
- 上昇後に下落する複合予想
- 同じ予想の再表明
- テーマ・業種の予想
- 話者不明のYouTube文字起こし
- 既に評価期間が終了した予想
- 現在も評価期間中の予想
- 想定より早く実現した予想

この試験セットを固定し、プロンプトや採点方法を変更するたびに同じデータで再検証します。

この「少数データによる一連動作を先に完成させ、媒体収集・複雑予想・複数アナリストへ順番に拡張する」方針を推奨します。

次に、当初ご希望だったループエンジニアリングを、実際の開発計画へ組み込みます。

ループの対象

すべてを毎回作り直さず、品質上重要な部分ごとに改善します。

原文整理

↓

予想抽出

↓

原文整合性検証

↓

予測対象解決

↓

市場実績評価

↓

レポート確認

↓

問題があった工程だけ修正・再実行



固定テストセット

同じ原文を繰り返し使用し、プロンプト変更前後の品質を比較します。

開発用テストセット

プロンプト改善時に使用します。

- 明確な予想
- 曖昧な予想
- 否定表現
- 条件付き予想
- 複合予想
- 再表明
- 話者不明
- テーマ予想
- 早期実現
- 予想ではない分析

最終確認用テストセット

プロンプト改善には直接使用せず、完成候補の性能確認に使います。開発用データだけに最適化されることを防ぎます。

評価項目

予想抽出

- 本来ある予想を抽出できた割合
- 抽出したものが本当に予想だった割合
- 原文にない数値や時期を補っていないか
- 否定表現を反対に解釈していないか
- 条件や本命シナリオを維持できているか
- 原文引用が正しいか

話者推定

- 本人発言の識別精度
- 司会者・他出演者との取り違え
- 判断不能な箇所を無理に確定していないか
- 推定根拠が原文・メタデータに存在するか

予測対象解決

- 原文で明示された対象を優先しているか
- 発言時点より後の情報を使っていないか
- ティッカーや銘柄コードが正しいか
- テーマと代理指標が対応しているか
- 結果を見て有利な銘柄を選んでいないか

Python評価

- 開始価格が正しいか
- 営業日調整が正しいか
- 株式分割を処理できているか
- 方向判定が正しいか
- 期限内到達と時点予想を区別できているか
- 早期実現を正しく評価できているか
- 同じ予想を二重計上していないか

ループの役割

作成役

プロンプト、仕様、コードを改善します。

批判役

書き直さず、問題だけを抽出します。

- 見逃し
- 捏造

- 矛盾
- 後知恵
- 評価上の偏り
- 再現できない判断
- 実装と仕様の不一致

統合役

指摘を無条件で採用せず、根拠を確認して修正案へ反映します。

高性能モデルを使用し、可能であれば作成役と批判役で異なるモデルを使います。ただし、モデル名は設定で変更可能にします。

修正履歴

プロンプトや評価方法を変更した場合は、次を保存します。

変更前バージョン
変更後バージョン
変更理由
発見された問題
修正内容
影響する処理
再実行が必要なデータ
テスト結果
採用・不採用



仕様変更は `DECISION_LOG.md`、将来機能は

`FUTURE_ROADMAP.md`、未解決問題は

`OPEN_QUESTIONS.md` へ分けます。

差分再実行

予想抽出プロンプトだけを変更した場合、市場価格データまですべて再取得する必要はありません。

- 変更されたプロンプトが影響するデータを特定
- 該当工程以降だけを再実行
- 変更前後の結果を比較
- 既存結果を即時削除せず、旧バージョンとして保持

これをPythonが管理します。

暫定的な合格基準

MVPでは次を目標にします。

- 原文引用の存在確認：100%
- Pythonの固定テスト：100%
- 原文にない数値・時期の追加：0件
- 話者不明を無理に本人認定：0件
- 重大な否定表現の取り違え：0件
- 予想抽出の見逃し：開発用テストで5%以下
- 予想ではない発言の誤採用：開発用テストで5%以下
- 重大な未解決不具合：0件
- 同じ重大指摘が2回連続で発生しない
- 最終確認用テストでも大幅な精度低下がない

数値基準は試験データ数が増えた段階で再検討します。少数データで95%などの数字だけを過信せず、具体的な誤りの内容も確認します。

終了条件

次をすべて満たした時点で、その工程の改善ループを終了します。

- 合格基準を満たした
- 重大な新規指摘が2回連続で出ていない
- 仕様と実装と説明書が一致している
- 最終確認用データでも再現できた
- 改善によって別の項目が悪化していない
- 残る問題が `FUTURE_ROADMAP.md` または `OPEN_QUESTIONS.md` に説明付きで記録されている

これにより、「もっと良くして」を無制限に繰り返すのではなく、測定可能な改善ループにできます。

ループエンジニアリングによる改善は今回は採用を見送り

ます。将来課題としてコードリファクタリングや機能追加をする際に、ループエンジニアリングによる改善を行いたいと思います。

承知しました。今回のMVP開発では、ループエンジニアリングを正式な開発工程として採用しません。

今回見送るもの

- 複数AIによる作成・批判・統合の反復
- 改善指摘がなくなるまでの自動反復
- プロンプト性能の継続的な比較ループ
- 評価結果に応じた自動的なコード再修正
- 開発用・最終確認用データを使った反復最適化

通常の実装、テスト、不具合修正は行いますが、それを「ループエンジニアリング」として自動化・体系化はしません。

引き続き採用するもの

以下はループエンジニアリングではなく、アプリの通常機能・品質管理として維持します。

- 低確信度・高重要度の予想に対するAIダブルチェック
- 予測対象解決の提案・検証・不一致時の裁定
- JSON Schema検証
- Pythonの単体テスト
- 原文引用の存在確認
- プロンプトと評価方法のバージョン管理
- エラーログと処理履歴
- 仕様書と取扱説明書
- MVP完成時の受入確認

`FUTURE_ROADMAP.md` への記載内容

一言のメモではなく、次の内容で詳細に残します。

FUT-XXX：ループエンジニアリングによる継続的改善

背景・目的

本アプリは、AIプロンプト、Pythonによる市場評価、予測対象の市場動向、AIによる市場動向の予測、AIによる市場動向の予測、複数の媒体からの情報収集など、多数の処理から構成される。将来的な機能追加や大規模なコードリファクタリングでは、一度の修正だけでは既存機能への影響を十分に確認できない可能性がある。

そのため、将来の大規模変更時には、作成、批判、検証、修正のループを繰り返すことで、一定の終了条件まで反復するループエンジニアリングを導入する。

適用対象

- ・大規模なコードリファクタリング
- ・データモデルの変更
- ・新しい情報媒体の追加
- ・市場データ取得元の追加・変更
- ・予想抽出プロンプトの大幅な変更
- ・評価方法やスコア計算式の変更
- ・複数アナリスト統合方法の変更
- ・処理速度やコストの大規模改善

軽微な文言修正や単純な不具合修正には原則として適用しない。

想定する処理

1. 変更前の動作と評価結果を基準値として保存する
2. 作成担当AIまたは開発者が変更を実施する
3. 別の高性能AIが欠陥・矛盾・後方互換性をレビューする
4. 固定テストデータと自動テストを実行する
5. 指摘の採否を判断して修正する
6. 変更前後の結果を比較する
7. 終了条件を満たすまで、上限回数内で反復する
8. 最終結果と残存課題を記録する

必要な準備

- ・固定された回帰テストデータ
- ・変更前後の比較機能
- ・プロンプトとコードのバージョン管理
- ・テスト結果の保存
- ・AIレビュー結果の構造化形式
- ・反復回数と処理コストの記録
- ・途中状態から再開できる仕組み

導入条件

以下のいずれかに該当する場合に導入を検討する。

- ・複数機能へ影響する変更を行う
- ・通常のレビューだけでは影響範囲を判断しにくい
- ・同種の不具合が繰り返し発生している
- ・プロンプト変更による精度低下を検出する必要がある
- ・既存の評価結果との互換性確認が必要である

終了条件

- ・ 重大なテスト失敗がない
- ・ 仕様と実装と説明書が一致している
- ・ 変更前に正常だった主要機能が維持されている
- ・ 重大な新規指摘が一定回数連続で発生していない
- ・ 反復回数の上限に達していない
- ・ 残存課題が記録されている

リスク・注意点

- ・ 同じAIによる反復で誤った前提が強化される可能性がある
- ・ 処理時間とAI利用量が増加する
- ・ テストデータへの過剰適合が起こり得る
- ・ 変更の目的より文章やコード量だけが増える可能性がある
- ・ 終了条件が曖昧だと反復が終わらない

現時点で見送る理由

初期開発段階では、まず一連の基本機能を完成させることを優先。ループ機構自体を実装するとアプリと開発工程がさらに複雑になり、MVP完成までの工数が増えるため、今回は採用しない。

今回の開発計画は「仕様確定→MVP実装→通常テスト→不具合修正→受入確認」という分かりやすい工程に変更します。

ok

次は技術構成を決めます。初期版はWindows上で動くPython製のターミナルアプリとし、Webアプリにはしません。

推奨技術構成

用途	技術
ターミナル画面	Python + Typer + Rich
データベース	SQLite
データ検証	Pydantic / JSON Schema
DB操作	SQLAlchemy
DB更新管理	Alembic
市場データ計算	pandas または Polars

用途	技術
グラフ	Plotly
Markdown生成	Jinja2
設定	YAML
APIキー等	環境変数または <code>.env</code>
テスト	pytest
Windows実行ファイル 化	PyInstaller等

開発中はPythonとして実行し、MVP完成時にWindows用の実行ファイルを作ります。

コード構成

```
src/
├ cli/                ターミナル画面
├ workflow/           進捗・次の行動判定
├ source_ingestion/   原文取込み
├ ai_outputs/         AI出力検証
├ forecasts/          予想構造化
├ targets/            予測対象・代理指標
├ market_data/        市場データ取得
├ evaluation/         的中評価
├ reporting/          表・グラフ・Markdown
├ persistence/        SQLite
└ configuration/      設定読み込み
```

「次何をすべきか」の判定は `workflow` に集約し、各機能が独自に処理順序を判断しない設計にします。

市場データ取得は差し替え可能にする

```
market_data/
├ jqunits_provider
├ alpha_vantage_provider
├ yfinance_provider
├ fred_provider
└ csv_provider
```


評価処理は、どの取得元を使ったかに依存しない
共通形式で受け取ります。

```
date
open
high
low
close
adjusted_close
volume
dividend
currency
provider
retrieved_at
```



MVPの取得方法案

費用と実装難度を抑えるなら、次の構成が現実的です。

- 株式・ETF・指数・為替：`yfinance`
- 経済指標：FRED
- 取得できない対象：CSV手動取込み
- 日本株の精度・安定性を高める場合：J-Quantsを追加
- 海外市場を安定運用する段階：Alpha Vantage等を追加

`yfinance` だけに固定せず、取得失敗時に別取得元またはCSVへ切り替えられるようにします。

APIキー管理

APIキーをObsidianのMarkdownやGit管理対象へ直接書きません。

```
JQUANTS_API_KEY
ALPHA_VANTAGE_API_KEY
FRED_API_KEY
```



設定の有無はターミナルで確認しますが、キー本体は表示しません。

実行ファイル

最終的には、例えば次の形式にします。



起動時にObsidian Vaultの場所を設定し、それ以降は設定ファイルから読み込みます。パスをコードへ直接書き込まないため、Vaultを移動しても再設定できます。

次に決める必要があること

市場データについて、次のどちらを重視するかで初期実装が変わります。

1. 無料・簡単さを優先

MVPは `yfinance` + FRED + CSVで開始し、J-Quants等は後から追加する。

2. 安定性・公式データを優先

最初からJ-Quants等のAPI契約・キー取得を前提にする。

推奨は1です。取得元を差し替え可能にしておけば、評価ロジックを作り直さずに後から公式・有料データへ移行できます。

1. 無料・簡単さを優先

承知しました。MVPでは「無料・簡単さ優先」で確定します。

初期の市場データ構成

- 株式・ETF・指数・為替：`yfinance`
- 経済指標：FRED
- 自動取得できないデータ：CSV手動取込み
- J-Quants、Alpha Vantage等：初期版では使わない
- 有料API：前提にしない

取得失敗時

`yfinance` で取得できない場合、AIやPythonが価格を推測することはありません。

1. ティッカー・期間・通貨を再確認
2. 再取得
3. それでも失敗した場合はCSV取込みを案内
4. CSVもなければdata_unavailable



ターミナルに、必要なCSVの形式と保存先を表示します。

データ品質確認

Pythonが次を自動検査します。

- 日付の重複
- 欠損期間
- 通貨
- 異常な価格変動
- 株式分割
- 調整前・調整後価格の混同
- 開始日・終了日の営業日調整
- 取得元と取得日時

異常がある場合は評価を強行せず、

`needs_data_review` とします。

将来の移行

市場データ取得部分は共通インターフェースにするため、後からJ-Quants等を追加しても、予想抽出・採点・レポート機能を作り直す必要はありません。

有料・公式データへの移行は、取得不能やデータ品質問題が増えた場合の将来課題として、背景、導入条件、候補API、変更箇所、完了条件まで

`FUTURE_ROADMAP.md` へ記録します。

OK

次に、最終的なレポートとグラフの構成を定めま
す。Obsidianで読みやすいMarkdownを中心に
し、詳細確認用としてCSVとHTMLも出力しま
す。

出力形式

形式	用途
Markdown	Obsidianで日常的に確認
CSV	全予想の一覧・Excel等での確認
PNG	Obsidian内に表示するグラフ
HTML	フィルター可能な詳細グラフ
JSON	AI処理・再計算用の構造化データ

分析対象者別レポート

```
analyst_summary.md
active_forecasts.md
all_forecasts.md
completed_forecasts.md
performance_by_field.md
performance_by_horizon.md
analysis_quality.md
research_coverage.md
limitations.md
```



`analyst_summary.md` には次をまとめます。

- 分析対象者の概要
- 評価済み予想数
- 現在評価中の予想数
- 方向・時期・程度の成績
- 具体性・検証可能性
- 得意・不得意分野
- 得意・不得意な予想期間
- 現在有効な主要予想
- 調査網羅性
- サンプル不足・評価上の注意

現在の上昇候補レポート

```
current_opportunities.md
```



予測対象ごとに次を表示します。

- 上昇予想人数
- 下落予想人数
- 過去成績で重み付けした方向
- 予想期間
- 数値予想がある場合の集計
- 見解の一致度
- 各アナリストの該当分野成績
- 現在の途中経過
- 代理指標依存度
- 根拠となる発言
- 反対意見
- 注意事項

グラフ

予想タイムライン

- 横軸：時期
- 縦軸：分析対象者
- 選択条件：予測対象
- 上昇：緑＋上向き矢印
- 下落：赤＋下向き矢印
- 横ばい：灰色＋横向き矢印
- 色の濃さ：予想された程度
- 点の大きさ：予想の具体性
- 枠線：証拠強度
- 線の長さ：予想期間

色だけに依存せず、矢印と文字も併用します。

予測対象ヒートマップ

- 縦軸：予測対象
- 横軸：時期
- 分析対象者：フィルター
- セルの色：上昇・下落
- セル内記号：方向
- ツールチップ：発言者、予想期限、根拠

能力マトリクス

	短期	中期	長
個別株	成績	成績	

	短期	中期	長
指数	成績	成績	
為替	成績	成績	
コモディティ	成績	成績	

サンプル不足のセルは点数を表示せず、「不足」とします。

3次元グラフは採用しない

当初の、

- 第1軸：時期
- 第2軸：予測対象
- 第3軸：分析対象者

という情報は維持しますが、1枚の3次元グラフにはしません。

- 時期×分析対象者のタイムライン
- 時期×予測対象のヒートマップ
- 分析対象者の選択フィルター

へ分解します。その方が実際の比較と操作がしやすくなります。

MVPの出力

最初のMVPでは次に限定します。

- 全予想CSV
- 全予想Markdown
- 現在有効な予想Markdown
- アナリスト概要Markdown
- 調査網羅性Markdown
- 静的タイムラインPNG
- 評価不能・未解決一覧

複数アナリストが蓄積した段階で、上昇候補ランキング、ヒートマップ、インタラクティブHTMLを追加します。これらは完成版の必須機能として開発計画に残します。

OK

次に、機能以外の品質要件を定めます。

利用環境

- Windows 11を正式な初期対応環境とする
- 個人・単一ユーザーでの利用を前提とする
- Obsidian Vault内へデータとレポートを保存する
- Webサーバーや常時起動サービスは使用しない
- 基本操作は日本語のターミナル画面とする
- インターネット接続は、AI検索と市場データ取得時に使用する

他のOS対応は、詳細説明付きで

`FUTURE_ROADMAP.md` へ記録します。

データ保全

- `raw` 原文は上書き・削除しない
- AI整理結果は別ファイルへ保存する
- SQLiteへの取込みはトランザクション単位で行う
- 途中で失敗した場合は中途半端なデータを確定しない
- 同じ処理を再実行しても重複登録しない
- DB更新前に自動バックアップを作る
- DB構造変更前にもバックアップを作る
- 古い評価結果とプロンプトバージョンを追跡できる

バックアップ

```
_system/backups/  
├ database/  
├ configuration/  
└ mappings/
```



バックアップには次を記録します。

- 作成日時

- 作成理由
- アプリバージョン
- DBバージョン
- 復旧方法

原文ファイルを含むVault全体のバックアップは、Obsidian側の同期・バックアップ方法と分けて取扱説明書で案内します。

障害復旧

エラー時には、単に処理を終了せず次を表示します。

失敗した処理

失敗理由

影響する予想・ファイル

正常に完了した範囲

再実行してよいか

次に行う操作

関連ログ



ターミナルの「エラー・未解決事項を表示」から確認できるようにします。

再現性・監査性

各結果から次を確認できるようにします。

- 使用した原文
- 原文の該当位置
- 使用したプロンプト
- 使用したAIモデル
- AI出力
- Python評価式
- 市場データ取得元
- 市場データ取得日時
- 評価用指標の選定理由
- 評価基準日
- アプリバージョン

処理速度

AI処理時間と外部API待ち時間は保証対象にできないため、次を重視します。

- ターミナルの進捗表示は数秒以内

- ローカルの一覧表示は待たされないこと
- 市場データはキャッシュし、同じ範囲を毎回取得しない
- Markdown・CSVは必要なものだけ再生成できる
- 大量処理は進捗件数を表示する
- 処理を中断して後から再開できる

具体的な秒数目標は、MVPで実データ量を確認してから決めます。

想定データ量

初期設計では、少なくとも次を扱える構造にします。

- 分析対象者：数十人
- 情報源：数万件
- 予想表明：数万～十万件
- 複数年分の市場日足データ

最初からすべてを高速処理する必要はありませんが、Markdownファイルだけを全件走査しないようSQLiteの索引を利用します。

秘密情報

- APIキーをMarkdownへ保存しない
- APIキーをログへ出力しない
- Gitへ登録しない
- ターミナルへキー本体を表示しない
- 設定確認では「設定済み／未設定」だけを表示する

外部公開

- レポートを自動公開しない
- SNSやWebサイトへの投稿機能を持たせない
- 外部共有用の機能をMVPに含めない
- すべてローカルの個人利用を前提とする

ただし、CursorやChatGPTで処理した原文は、それぞれのAIサービスへ入力される点を取扱説明書に明記します。

操作性

- 色だけで状態を示さない
- 専門用語には説明を付ける
- エラーメッセージに解決方法を含める
- 次の推奨操作を常に表示する
- AI処理かPython処理かを明記する
- ファイルパスはコピー可能な形式で表示する
- 通常利用でSQLiteを直接編集する必要がない
- 通常利用で21種類のプロンプト名を暗記する必要がない

これらを非機能要件として仕様書へ含めます。

OK

13m 45s考えました >

仕様・計画書一式が完成しました。合意した内容を17個のMarkdown文書に整理し、以下を反映しています。

- AIプロンプトとPython処理の明確な役割分担
- 予想の抽出・対象解決・市場実績評価・現在予想の集約方法
- ターミナルによる「次にすべきこと」の案内設計
- README、スタートアップガイド、取扱説明書、詳細仕様書
- 判断記録、未決事項、MVP開発計画
- ループエンジニアリングを含む、詳しい将来ロードマップ
- 無料・簡単さを優先した市場データ取得方針

主なファイルはこちらです。

- [文書一式をダウンロード \(ZIP\)](#)
- [📄 README.md](#)
- [📄 SYSTEM_SPECIFICATION.md](#)
- [📄 STARTUP_GUIDE.md](#)
- [📄 FUTURE_ROADMAP.md](#)

ZIPの展開検査と文書構造の整合性確認も完了しています。次の段階は、残っている未決事項を確

定したうえで、MVPの実装用リポジトリを作る工程です。

ありがとう。実装はCursor上で行います。Cursorに渡すべき依頼文と資料一式を準備してほしい。合わせてこのチャットの全文も送ろうと思いますが良いですか

思考中